Deep Behavior Analysis Report v1

Purpose

This analysis checks whether the tone effect from the current 2,700-run experiment remains after accounting for output behavior:

- output length (`completion_tokens`)
- code-only compliance (`likely_code_only`)
- code extraction/parsing failure (`extraction_failed`)
- runtime (`duration_seconds`)

The goal is to deeper understand the reason why neutral prompts perform better.

Data

- Models: 3 (`gemma2-9b`, `llama3.1-8b`, `qwen2.5-7b`)
- Problems: 50
- Prompt templates: 18
- Total runs: 2,700
- Prompt factors: `Tone`, `Role_framing`, `Format`
- Corrected result files:
  - `results/gemma2-9b_fixextractor.jsonl`
  - `results/llama3.1-8b_fixextractor.jsonl`
  - `results/qwen2.5-7b_fixextractor.jsonl`

Model Performance

Model	Runs	Pass rate	Avg. tests passed	Avg. duration	Avg. completion tokens	Code-only rate
qwen2.5-7b	900	13.4%	17.0%	18.59s	653	26.7%
gemma2-9b	900	9.4%	14.0%	19.56s	601	25.9%
llama3.1-8b	900	5.3%	8.9%	16.33s	562	23.0%

Descriptive Results by Tone

Tone still changes behavior strongly.

Tone	Pass rate	Avg. tests passed	Avg. completion tokens	Avg. duration	Code-only rate	Invalid-Python rate	Runtime failure rate
neutral	11.8%	16.9%	413	12.62s	69.4%	0.2%	35.0%
polite	8.1%	11.3%	716	21.47s	1.6%	0.0%	52.3%
aggressive	8.3%	11.7%	687	20.40s	4.6%	0.4%	49.2%

The largest behavioral difference remains code-only compliance. Neutral prompts produce much shorter outputs and much more code-only output. Polite and aggressive prompts produce longer responses with more extra prose/wrapper text and much lower code-only compliance.

Regression Strategy

The main robustness check uses fixed-effect OLS models with model and problem fixed effects:

```
outcome ~ Tone + Role_framing + Format + model fixed effects +
problem fixed effects + behavior controls
```

Two outcomes are analyzed:

- `passed_int`: exact pass/fail
- `tests_fraction`: fraction of tests passed

The baseline model has prompt factors only. Follow-up models add behavior controls:

- `M0_prompt_only`
- `M1_length`: adds log completion tokens
- `M2_code_only`: adds likely code-only response
- `M3_invalid_python`: adds invalid-Python response after fixed extraction
- `M4_runtime`: adds log runtime
- `M5_all_behavior_controls`: adds all behavior controls

Logistic GLM models were also run for `passed_int` as a robustness check.

Key Finding: The Neutral Effect Mostly Disappears After Behavior Controls

With aggressive tone as the reference category:

Outcome	Model	Neutral coefficient	p-value	Interpretation
pass/fail	Prompt factors only	+3.4 pp	0.0050	Neutral is significantly better before behavior controls
pass/fail	+ output length	-2.7 pp	0.0859	Neutral no longer significant
pass/fail	+ code-only	+0.8 pp	0.5996	Neutral no longer significant
pass/fail	+ invalid Python	+3.4 pp	0.0051	Invalid-Python failures do not explain the tone effect
pass/fail	+ runtime	-2.2 pp	0.1434	Neutral no longer significant
pass/fail	all behavior controls	-2.8 pp	0.0877	Neutral no longer significant

For `tests_fraction`, the same pattern appears:

Outcome	Model	Neutral coefficient	p-value	Interpretation
tests fraction	Prompt factors only	+5.3 pp	0.0001	Neutral is significantly better before behavior controls
tests fraction	+ output length	-1.9 pp	0.2710	Neutral no longer significant
tests fraction	+ code-only	+1.5 pp	0.3980	Neutral no longer significant

Outcome	Model	Neutral coefficient	p-value	Interpretation
tests fraction	+ invalid Python	+5.2 pp	0.0001	Invalid-Python failures do not explain the tone effect
tests fraction	+ runtime	-1.5 pp	0.3843	Neutral no longer significant
tests fraction	all behavior controls	-2.5 pp	0.1882	Neutral no longer significant

This supports a mediation-style interpretation: the observed tone effect is largely explained by output behavior, especially verbosity/runtime and code-only compliance.

Behavior Controls Matter

In separate fixed-effect models:

- longer outputs are associated with lower correctness
- longer runtime is associated with lower correctness
- code-only responses are associated with higher correctness
- invalid-Python responses are rare and do not explain the tone effect

For `tests_fraction`:

Control	Coefficient	p-value
log completion tokens	-12.7 pp	< 0.001
code-only response	+5.8 pp	0.0027
invalid Python after extraction	-7.3 pp	0.1448
log runtime	-13.8 pp	< 0.001

When all behavior controls are included together, completion tokens and duration remain highly correlated (`r = 0.985`). The all-controls model should therefore be treated as a robustness check, not as a clean causal decomposition of length versus runtime.

Logistic Robustness Check

The logistic GLM results for pass/fail show the same qualitative pattern:

- baseline neutral-tone effect is significant
- after controlling for output length or runtime, the neutral-tone coefficient is no longer significant
- after all behavior controls, neutral tone is no longer conventionally significant

This supports the OLS findings and reduces concern that the result is only an artifact of using a linear probability model.

Model-Specific Pattern

After fixing extraction, neutral tone is best for all three models by average tests passed.

Model	Best tone by tests fraction	Neutral tests fraction	Neutral code-only rate
gemma2-9b	neutral	18.0%	77.7%
llama3.1-8b	neutral	11.8%	51.7%
qwen2.5-7b	neutral	21.1%	79.0%

This is an important update from the previous report. Before the extractor fix, Llama appeared to be a caveat because many neutral plain-code responses were missed. After correction, Llama also shows its best average tests fraction under neutral prompts.

The safer paper wording is:

> Neutral tone produced the strongest overall result across the three tested models, and the advantage is largely mediated by output behavior.

Difficulty Split

Problems were split into hard/medium/easy bands based on average test performance across all runs.

Difficulty	Neutral tests fraction	Polite tests fraction	Aggressive tests fraction
hard	0.6%	0.1%	0.1%
medium	12.2%	5.6%	7.0%
easy	37.8%	27.9%	27.7%

Neutral tone is most useful in the medium and easy bands. On the hardest problems, all models remain near zero, so prompt tone cannot rescue tasks that are beyond the model/problem setup.

Error Modes

The dominant failure categories are wrong answers and missing function/name errors.

Tone	Most common failure pattern
neutral	wrong answer, then name-not-defined
polite	name-not-defined, then wrong answer
aggressive	name-not-defined, then wrong answer

Invalid-Python responses are rare after the fixed extractor:

- neutral: 0.2%
- polite: 0.0%
- aggressive: 0.4%

This means the main behavioral mechanism is not invalid syntax. The stronger mechanism is verbosity and evaluator compliance: neutral prompts are much more likely to return clean code-only responses.

Interpretation

> Prompt tone affects coding performance mainly by changing output behavior. Neutral prompts produce shorter, faster, and more code-focused responses, with better performance. Polite and aggressive prompts encourage longer explanatory responses, which increases latency and reduces code-only compliance.

Notes

- Completion tokens and runtime are highly correlated, so their individual coefficients are unstable when included together.
- Exact pass rates remain low, so `tests_fraction` remains the more informative performance metric.